

Break the login

Raport scris

Nichita Iulia-Nicoleta, 341

Aprilie 2026

1 Introducere

Compania fictivă *AuthX* utilizează o aplicație web pentru a gestiona accesul la resurse sensibile al angajaților săi. Pentru a îndeplini această condiție, aplicația salvează datele utilizatorilor (Users), biletele stocate (Tickets) și monitorizează activitățile (Audit logs).

Un utilizator este recunoscut din momentul înregistrării în platformă prin *formularul de register*, iar apoi cu aceste date se poate loga prin *formularul de login*. Utilizatorii sunt înregistrați cu rolul fiecăruia, ANALYST sau MANAGER. Rolul de ANALYST permite vizualizarea și editarea propriilor bilete, precum și editarea profilului. Rolul de MANAGER are aceleași permisiuni ca cele de ANALYST la care se adaugă și vizualizarea activităților înregistrate. Indiferent de rol, utilizatorilor le este permis să își reseteze parola contului personal prin *formularul de resetpassword*, adăugând email-ul contului și adăugând noua parolă pe o pagină cu acest scop.

Arhitectură

Aplicația AuthX utilizează o arhitectură client-server clasică, în care frontend-ul comunică cu backend-ul printr-un API REST. Frontend-ul aplicației este compus din pagini HTML statice, stilizate cu CSS, care utilizează formulare pentru a primi datele introduse de utilizatori în câmpurile dedicate. Schimbul de date dintre interfață și server se realizează prin logica de fetch, utilizând formatul JSON și metodele standard HTTP: GET, POST, PUT și DELETE.

Backend-ul are rolul de a procesa cererile primite, de a realiza logica funcționalităților platformei și de a gestiona accesul la baza de date prin metodele enumerate anterior. Acesta expune un set de endpoint-uri REST prin care frontend-ul poate efectua operații asupra entităților aplicației: Users, Tickets, Audit_logs. Baza de date aleasă este SQLite, datorită simplității, portabilității și performanței, fiind o soluție de tip *zero-configuration*, stocată într-un singur fișier, fără a necesita un server dedicat.

procesul de dezvoltare a unei aplicații, lansare a unui produs doar cu functionalitatile necesare owasp mapping - identificarea, analiza, asocierea vulnerabilităților cunoscute in aplicatie cu cele enumerate de OWASP demonstrare poc - demonstrarea unei vulnerabilitati prezentate de un anumit produs, sistem printr-un atac simulat idor (Insecure Direct Object Reference) - vulnerabilitate de acces a sistemului in care un user poate accesa printr un input un obiect fara autorizare stack trace - raport ce arata declansarea unei erori in cadrul unui program prin functiile apelate

xss (Cross-Site Scripting) - vulnerabilitate web prin care atacatorii pot adauga cod JS malitios in website uri (cand un user acceseaza o pagina se ruleaza acest JS file) hijacking - atac prin care atacatorul preia controlul asupra sesiunii user-ului, contului csrf (Cross-Site Request Forgery) - se refera la un site malicious care se foloseste de faptul ca ai cookie-ul setat in sesiune cu token si cand dai click pe site iti incarca un cod pe care nu il vezi (de ex apeleaza api /changeemail) si functioneaza ptc cererea se face cu cookie ul user-ului victima injection prevention (limit results for search) - tehnici, tehnologii, standarde web utilizate pt a impiedica atacatorii sa injecteze cod malitios pt a extrage informatii / a prelua accesul, prin input-uri rendering raw HTML - se refera la a nu pune direct input-uri in html ptc atacatorii pot pune cod js si pot avea acces la info sensibile, cookie de ex